

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

8-mashg’ulot. Pythonda ro‘yxatlar va ular bilan ishlovchi metodlar.

O‘quv savollari:

1. Ro‘yxatlar va ularning qo‘llanilishi.
2. Ro‘yxatlarni yaratish usullari.
3. Ro‘yxatlar bilan ishlovchi metodlar.

1. Ro‘yxatlar va ularning qo‘llanilishi

Kirish

- Python dasturlash tilida ro‘yxat (list) — bu o‘zgaruvchan (mutable) ma'lumotlar turidir, ya'ni uning elementlarini o‘zgartirish mumkin.
- Ro‘yxatlar (list) — bu ma'lumotlar tuzilmasi bo‘lib, bir nechta turdagi ma'lumotlarni bir joyga jamlash imkonini beradi. Masalan, ro‘yxatga bir vaqtning o‘zida sonlar, matnlar, boolean (haqiqiy yoki yolg‘on) qiymatlar va boshqa turdagi ma'lumotlarni joylashtirish mumkin. Misol:

```
lst = [1, 'hello', True, 3.14]
```

Yuqoridagi ro‘yxatda bir nechta turdagi ma'lumotlar — son (1), matn ('hello'), boolean (True) va haqiqiy son (3.14) bir joyga jamlangan.

- Ro‘yxatlar tartiblangan ma'lumotlar tuzilmasi hisoblanadi, ya'ni ro‘yxatdagi har bir elementning o‘z o‘rni (indeksi) bor.
- Har bir elementga uning indeksiga qarab murojaat qilish mumkin.

Ro‘yxatlar va ularning qo‘llanilishi

Ro‘yxatlar juda foydali bo‘lib, quyidagi holatlarda ishlatiladi:

- **Ma'lumotlar to‘plamini saqlash:** Ro‘yxatlar turli xil ma'lumotlarni bir joyda saqlash uchun ishlatiladi. Masalan, sonlar, satrlar, bo‘lishi mumkin bo‘lgan boshqa ro‘yxatlar va hattoki boolean (true/false) qiymatlari.
- **Tartiblangan ma'lumotlar:** Ro‘yxatlar elementlarni tartiblangan (indekslangan) holda saqlaydi, bu esa elementlarga oson murojaat qilish imkonini beradi.
- **Dinamik o‘lcham:** Ro‘yxat o‘lchami (yoki uzunligi) dinamik bo‘lib, yangi elementlarni qo‘shish yoki mavjudlarini o‘chirish mumkin.
- **Boshqa ma'lumot tuzilmalarini saqlash:** Ro‘yxatlar ichida boshqa ro‘yxatlar, ma'lumotlar bazasi yozuvlari, obyektlar va boshqa ma'lumot tuzilmalarini saqlash mumkin.

Misol:

Ro‘yxat yordamida talabalarning nomlari ro‘yxatini saqlash:

```
students = ['Ali', 'Vali', 'Zuhra', 'Salim']  
print(students)
```

Yuqoridagi misolda `students` ro‘yxati orqali bir nechta talabaning nomlari saqlanadi va ro‘yxatga qo‘shish, o‘chirish yoki elementlarga murojaat qilish mumkin.

Amaliy mashqlar

1. Quyidagi kodni ishlatganda, qanday natija chiqadi?

```
lst = [1, 2, 3]
print(lst[1])
```

Javob: 2

2. Quyidagi kodni ishlatganda, qanday natija chiqadi?

```
lst = ['apple', 'banana', 'cherry']
print(lst[-2])
```

Javob: banana

3. Quyidagi kodni ishlatganda, qanday natija chiqadi?

```
lst = [10, 20, 30, 40]
print(lst[1:3])
```

Javob: [20, 30]

4. Quyidagi kodni ishlatganda, qanday natija chiqadi?

```
lst = ['apple', 'banana', 'cherry', 'date']
lst[1:3] = ['kiwi', 'mango']
print(lst)
```

Javob: ['apple', 'kiwi', 'mango', 'date']

5. Quyidagi kodni ishlatganda, qanday natija chiqadi?

```
lst = [1, 2, 3, 4, 5]
lst[1:4] = ['a', 'b']
print(lst)
```

Javob: [1, 'a', 'b', 5]

2. Ro'yxatlarni yaratish usullari

Python dasturlash tilida ro'yxatlarni yaratishning bir necha usullari mavjud. Ro'yxatlarni yaratish uchun eng oddiy va qulay usul **kvadrat qavslar** `[]` yordamida amalga oshiriladi. Shuningdek, ro'yxatlarni yaratishda `list()` funksiyasidan ham foydalanish mumkin. Quyida har ikki usulni ko'rib chiqamiz.

1. Kvadrat qavslar yordamida ro'yxat yaratish

Ro'yxatni yaratishning eng oson usuli bu **kvadrat qavslar** `[]` yordamida ro'yxatga kerakli elementlarni yozishdir.

Misol:

```
lst = [1, 2, 3, 4, 5]
```

Yuqoridagi misolda ro'yxatni yaratish uchun **kvadrat qavslar** ishlatilgan va ro'yxatga **5 ta element** qo'shilgan.

2. `list()` funksiyasi yordamida ro'yxat yaratish

Ro'yxatni yaratish uchun `list()` funksiyasidan ham foydalanish mumkin. Bu funksiya, odatda, boshqa ma'lumotlar tuzilmalarini (masalan, satrlar yoki tuples) ro'yxatga aylantirish uchun ishlatiladi.

Misol:

```
lst = list('hello')
print(lst) # ['h', 'e', 'l', 'l', 'o']
```

Yuqoridagi misolda `list()` funksiyasi yordamida satr `hello` ro'yxatga aylantirilgan. Natijada, har bir harf alohida element sifatida ro'yxatda joylashgan.

3. Bo'sh ro'yxat yaratish

Bo'sh ro'yxatni yaratish uchun, faqat **kvadrat qavslar** ishlatiladi.

Misol:

```
lst = []
print(lst) # []
```

Bu ro'yxat dastlab bo'sh, lekin keyinchalik unga elementlar qo'shish mumkin.

Ro'yxatlarni yaratishning qo'shimcha usullari

- **`list()` funksiyasi bilan boshqa tuzilmalar (masalan, tuple) ro'yxatga aylantirish:**

```
tup = (1, 2, 3)
lst = list(tup)
print(lst) # [1, 2, 3]
```

- **list()** funksiyasi bilan satrni ro'yxatga aylantirish

```
satr = "hello"  
lst = list(satr)  
print(lst)
```

Natija:

```
['h', 'e', 'l', 'l', 'o']
```

- **range()** funksiyasi yordamida ro'yxat yaratish:

```
lst = list(range(5))  
print(lst) # [0, 1, 2, 3, 4]
```

Amaliy mashqlar

1. Quyidagi kodni ishlatganda qanday natija chiqadi?

```
lst = list('abc')  
print(lst)
```

Javob: ['a', 'b', 'c']

2. Quyidagi kodni ishlatganda qanday natija chiqadi?

```
lst = list(range(3, 7))  
print(lst)
```

Javob: [3, 4, 5, 6]

3. Quyidagi kodni ishlatganda qanday natija chiqadi?

```
lst = list('1234')  
print(lst)
```

Javob: ['1', '2', '3', '4']

4. Quyidagi ro'yxatni yaratish usulini ko'rsating:
1 dan 10 gacha bo'lgan juft sonlarni ro'yxatga joylashtirish.

Javob:

```
lst = list(range(2, 11, 2))  
print(lst)
```

Natija: [2, 4, 6, 8, 10]

3. Ro'yxatlar bilan ishlovchi metodlar

Python ro'yxatlar bilan ishlash uchun bir nechta qulay metodlarga ega. Ushbu metodlar ro'yxatlarni tahrirlash, ularga yangi elementlar qo'shish, o'chirish, tartiblash va boshqa amallarni bajarish imkonini beradi. Quyida ro'yxatlar bilan ishlovchi metodlar va ularning vazifalari keltirilgan.

Ro'yxatlar bilan ishlovchi metodlar jadvali

Metod nomi	Vazifasi
append()	Ro'yxatning oxiriga yangi element qo'shish.
insert()	Belgilangan indeksga yangi element qo'shish.
extend()	Boshqa ro'yxatni birinchi ro'yxatga qo'shish.
remove()	Ro'yxatdan birinchi uchragan elementni qiymatiga qarab o'chirish.
pop()	Ro'yxatdan belgilangan indeksdagi elementni o'chirib, qaytarish.
clear()	Ro'yxatdagi barcha elementlarni o'chirish.
index()	Ro'yxatda elementning indeksini topish.
count()	Ro'yxatda ma'lum bir element nechta marta uchrashini hisoblash.
sort()	Ro'yxatni o'sish tartibida tartiblash.
reverse()	Ro'yxatdagi elementlarni teskari tartibda joylashtirish.
copy()	Ro'yxatning nusxasini olish.
list()	Har qanday boshqa ma'lumotlar turini ro'yxatga aylantirish.

Metodlar va ularning vazifalari

1. **append()** metodi

Vazifasi:

append() metodi ro'yxatning oxiriga yangi element qo'shadi.

Sintaksis:

```
lst.append(element)
```

Misol:

```
lst = [1, 2, 3]
lst.append(4)
print(lst) # Natija: [1, 2, 3, 4]
```

2. **insert()** metodi

Vazifasi:

insert() metodi ro'yxatda belgilangan indeksga yangi element qo'shadi.

Sintaksis:

```
lst.insert(index, element)
```

Misol:

```
lst = [1, 2, 3]
lst.insert(1, 4)
```

```
print(lst) # Natija: [1, 4, 2, 3]
```

3. **extend()** metodi

Vazifasi:

extend() metodi bir ro'yxatni boshqa ro'yxat bilan kengaytiradi (ya'ni, birinchi ro'yxatning oxiriga ikkinchi ro'yxatdagi barcha elementlarni qo'shadi).

Sintaksis:

```
lst.extend(boshqa_lst)
```

Misol:

```
lst1 = [1, 2, 3]
lst2 = [4, 5, 6]
lst1.extend(lst2)
print(lst1) # Natija: [1, 2, 3, 4, 5, 6]
```

4. **remove()** metodi

Vazifasi:

remove() metodi ro'yxatdan birinchi uchragan belgilangan elementni o'chiradi.

Sintaksis:

```
lst.remove(element)
```

Misol:

```
lst = [1, 2, 3, 2, 4]
lst.remove(2)
print(lst) # Natija: [1, 3, 2, 4]
```

Izoh:

remove() faqat birinchi uchragan elementni o'chiradi va agar element ro'yxatda bo'lmasa, xato beradi.

5. **pop()** metodi

Vazifasi:

pop() metodi ro'yxatdan belgilangan indeksdagi elementni o'chirib, uni qaytaradi. Agar indeks ko'rsatilmasa, oxirgi elementni olib tashlaydi.

Sintaksis:

```
element = lst.pop(index)
```

Misol:

```
lst = [1, 2, 3]
```

```
element = lst.pop(1)
print(lst) # Natija: [1, 3]
print(element) # Natija: 2
```

Izoh:

`pop()` ro'yxatni o'zgartiradi va o'chirilgan elementni qaytaradi.

6. `clear()` metodi

Vazifasi:

`clear()` metodi ro'yxatdagi barcha elementlarni o'chiradi.

Sintaksis:

```
lst.clear()
```

Misol:

```
lst = [1, 2, 3]
lst.clear()
print(lst) # Natija: []
```

Izoh:

Bu metod ro'yxatdagi barcha elementlarni o'chiradi va ro'yxatni bo'sh holga keltiradi.

7. `index()` metodi

Vazifasi:

`index()` metodi ro'yxatda belgilangan elementning indeksini qaytaradi.

Sintaksis:

```
index = lst.index(element)
```

Misol:

```
lst = [1, 2, 3, 4]
index = lst.index(3)
print(index) # Natija: 2
```

Izoh:

Agar element ro'yxatda mavjud bo'lmasa, **ValueError** xatosi yuz beradi.

8. `count()` metodi

Vazifasi:

`count()` metodi ro'yxatda ma'lum bir element nechta marta uchrashini hisoblaydi.

Sintaksis:

```
lst.count(element)
```

Misol:

```
lst = [1, 2, 2, 3, 2]
count = lst.count(2)
print(count) # Natija: 3
```

Izoh:

Bu metod ro'yxatdagi belgilangan elementlarning sonini hisoblaydi.

9. `sort()` metodi

Vazifasi:

`sort()` metodi ro'yxatni o'sish tartibida tartiblaydi.

Sintaksis:

```
lst.sort()
```

Misol:

```
lst = [4, 2, 3, 1]
lst.sort()
print(lst) # Natija: [1, 2, 3, 4]
```

Izoh:

`sort()` metodida `reverse=True` argumenti qo'shib, ro'yxatni kamayish tartibida tartiblash mumkin.

10. `reverse()` metodi

Vazifasi:

`reverse()` metodi ro'yxatdagi elementlarni teskari tartibda joylashtiradi.

Sintaksis:

```
lst.reverse()
```

Misol:

```
lst = [1, 2, 3, 4]
lst.reverse()
print(lst) # Natija: [4, 3, 2, 1]
```

Izoh:

`reverse()` metodida ro'yxatning tartibi o'zgartiriladi, lekin elementlar o'zgarishsiz qoladi.

11. `copy()` metodi

Vazifasi:

`copy()` metodi ro'yxatning yuzaki nusxasini yaratadi.

Sintaksis:

```
yangi_lst = lst.copy()
```

Misol:

```
lst = [1, 2, 3]
new_lst = lst.copy()
print(new_lst) # Natija: [1, 2, 3]
```

Izoh:

`copy()` metodida ro'yxat nusxasini yaratish mumkin, lekin bu metod faqat **yuzaki nusxa** yaratadi. Agar ro'yxatda ichki ro'yxatlar bo'lsa, ular o'zgarishlarga ta'sir qiladi.

12. `list()` funksiyasi

Vazifasi:

`list()` funksiyasi boshqa ma'lumotlar turini ro'yxatga aylantiradi (masalan, tuple, string, set).

Sintaksis:

```
lst = list(malumot_turi)
```

Misol:

```
tup = (1, 2, 3)
lst = list(tup)
print(lst) # Natija: [1, 2, 3]
```

Izoh:

`list()` funksiyasi tuple, set yoki boshqa iteratsiya qilinadigan tuzilmalarni ro'yxatga aylantirishda ishlatiladi.

Amaliy mashqlar:

1-misol:

Do'kon aravachasida ["olma", "banan"] mahsulotlari bor. Xaridor "uzum" ham qo'shmoqchi. Ro'yxatga qo'shib, natijani chiqaring.

Javob:

```
cart = ["olma", "banan"]
cart.append("uzum")
print(cart)
# Natija: ['olma', 'banan', 'uzum']
```

2-misol:

Mehmonlar ro'yxati ["Ali", "Vali", "Hasan"]. Siz Hasan oldiga "Dilshod"ni qo'shmoqchisiz. Natijani chiqarib bering.

Javob:

```

guests = ["Ali", "Vali", "Hasan"]
guests.insert(2, "Dilshod")
print(guests)
# Natija: ['Ali', 'Vali', 'Dilshod', 'Hasan']

```

3-misol:

Kutubxonada mavjud kitoblar ro'yxati ["Matematika", "Fizika"]. Sizning do'stingiz yana ["Tarix", "Biologiya"] kitoblarini olib keladi. Ularni qo'shib, umumiy ro'yxatni chiqaring.

Javob:

```

books = ["Matematika", "Fizika"]
new_books = ["Tarix", "Biologiya"]
books.extend(new_books)
print(books)
# Natija: ['Matematika', 'Fizika', 'Tarix', 'Biologiya']

```

4-misol:

Xaridlar ro'yxati ["non", "sut", "shakar", "tuz"]. Siz "shakar"ni olishdan voz kechdingiz. Ro'yxatdan olib tashlab, yangisini chiqaring.

Javob:

```

shopping = ["non", "sut", "shakar", "tuz"]
shopping.remove("shakar")
print(shopping)
# Natija: ['non', 'sut', 'tuz']

```

5-misol:

Talabalar baholari [56, 78, 90, 45, 90, 100].

- Eng past bahoni chiqarib tashlang (pop() bilan).
- Necha marta 90 bahosi uchraganini hisoblang.
- Qolganlarini kamayish tartibida tartiblang.

Javob:

```

grades = [56, 78, 90, 45, 90, 100]
grades.pop(grades.index(min(grades))) # eng kichikni o'chirish
print(grades) # [56, 78, 90, 90, 100]

print(grades.count(90)) # Natija: 2

grades.sort(reverse=True)
print(grades) # Natija: [100, 90, 90, 78, 56]

```

6-misol:

Ishchilar ro'yxati ["Sardor", "Lola", "Javohir", "Madina"].

- Ro'yxatning nusxasini oling.
- Nusxasini teskari aylantiring.

- Asl ro'yxatni bo'shating.

Javob:

```
workers = ["Sardor", "Lola", "Javohir", "Madina"]
copy_workers = workers.copy()

copy_workers.reverse()
print(copy_workers)
# Natija: ['Madina', 'Javohir', 'Lola', 'Sardor']

workers.clear()
print(workers)
# Natija: []
```

